# Smart Skin Analyzer

## Module d'analyse du visage par intelligence artificielle

## Acné · Rides · Taches · Pores

Guide de réalisation et explication détaillée des scripts
Projet IoT : miroir intelligent (Raspberry Pi 4 + ESP32 + caméra)

Version du 23/10/2026

# Sommaire


**1. Objectif et vue d'ensemble** **4**

1.1 Architecture du pipeline . . . 4

1.2 Lien avec le matériel du projet . . . 5

**2. Environnement de travail** **5**

2.1 Organisation des dossiers . . . 5

2.2 Logiciels . . . 6

2.3 Problèmes rencontrés et solutions . . . 6

2.4 Installation . . . 6

**3. Phase 1 : module d'analyse (vision classique)** **7**

3.1 config.py : paramètres . . . 7

3.2 face_regions.py : points du visage et zones de peau . . . 7

3.3 detectors.py : les quatre détecteurs classiques . . . 8

3.4 analyzer.py, visualize.py, main.py . . . 9

3.5 tests/test_synthetic.py : test par injection . . . 10

**4. Phase 2 : bases de données d'apprentissage** **11**

4.1 Bases étudiées et choix . . . 11

4.2 training/download_data.py . . . 12

**5. Modèle d'acné : YOLO11 sur ACNE04** **13**

5.1 prepare_acne_data.py : préparation en tuiles . . . 13

5.2 train_acne.py : entraînement . . . 13

5.3 Choix de la taille du modèle : mesures réelles . . . 14

5.4 Métriques d'évaluation . . . 14

**6. Modèle de rides : U-Net sur FFHQ-Wrinkle** **16**

6.1 Pourquoi la segmentation ? . . . 16

6.2 train_wrinkles.py : méthode en deux étapes . . . 16

6.3 Métriques . . . 17

**7. Intégration dans le miroir : dl_models.py** **19**

7.1 Rides : alignement FFHQ . . . 19

7.2 Acné : inférence par tuiles . . . 19

**8. Procédure complète pas à pas** **19**

8.1 Lancer et suivre un entraînement long . . . 20

**9. Partie matérielle : miroir, ESP32 et Raspberry Pi** **20**

9.1 Rôle de chaque composant . . . 20

9.2 Carte retenue : Arduino Uno + capteur Sharp IR . . . 21

9.2 bis Variante : câblage de l'ESP32 . . . 21

9.3 Montage du miroir . . . 22
9.4 Firmware de l'ESP32 (firmware/esp32_capteurs.ino) . . . 22
9.5 Installation du Raspberry Pi . . . 22
9.6 Programme du miroir (mirror/) . . . 23
9.6 bis Température de la peau : mesure en 2 temps . . . 23
9.7 Mise en service, étape par étape . . . 24

**10. Limites, éthique et suite du projet 24**

10.1 Limites . . . 24
10.2 Éthique et licences . . . 24
10.3 Prochaines étapes . . . 24
10.4 Références . . . 25

# 1. Objectif et vue d'ensemble

Ce module est la partie « intelligence » du miroir intelligent **Smart Skin Analyzer**. À partir d'une image du visage prise par la caméra du miroir, il détecte et mesure quatre critères visibles de l'état de la peau, puis produit un score et un rapport exploitables par l'interface du miroir et par la partie IoT.

| Critère | Ce qui est mesuré | Méthode retenue |
|---|---|---|
| Acné | Boutons (papules, pustules, comédons) : nombre, position, taille | **Apprentissage profond** : détecteur YOLO11 entraîné sur ACNE04 |
| Rides | Rides et ridules : masque au pixel près, longueur par zone | **Apprentissage profond** : segmentation U-Net entraînée sur FFHQ-Wrinkle |
| Taches | Taches pigmentaires (zones plus sombres, non rouges) : % de surface | Vision par ordinateur classique (contraste local dans l'espace L*a*b*) |
| Pores / trous | Pores dilatés, points noirs, petites cicatrices : densité par cm² | Vision par ordinateur classique (transformée morphologique black-hat) |

Le développement s'est fait en **deux phases** :

- **Phase 1 : prototype en vision classique.** Les quatre critères sont détectés par des règles de traitement d'image, sans entraînement. Cette phase a mis en place toute la chaîne (caméra, points du visage, zones, scores, affichage) et sert de solution de secours.
- **Phase 2 : modèles d'apprentissage profond.** Pour l'acné et les rides, deux vrais modèles sont entraînés sur des bases publiques annotées par des humains, puis évalués sur des images jamais vues pendant l'entraînement.

## 1.1 Architecture du pipeline

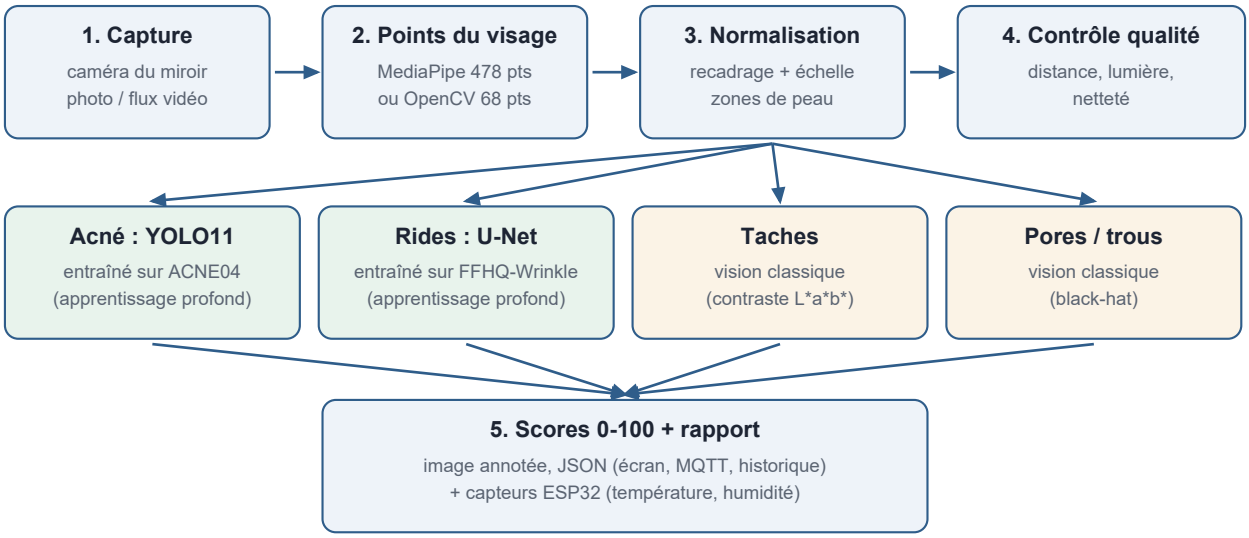



*Figure 1 : chaîne de traitement complète. En vert, les modèles d'apprentissage profond ; en orange, la vision classique.*

### 1.2 Lien avec le matériel du projet

| Matériel | Rôle pour ce module |
|---|---|
| Raspberry Pi 4 | Exécute l'analyse (modèles au format ONNX lus par OpenCV) et l'interface |
| Caméra Pi / webcam HD | Fournit l'image du visage ; 1080p recommandé pour les pores |
| VL53L0X (distance) | Déclencher l'analyse quand le visage est à la bonne distance (≈ 30-40 cm) |
| Bande LED / LED RGB | Éclairage frontal uniforme : réduit les ombres prises pour des taches |
| MLX90614, DHT22 (ESP32) | Mesures complémentaires ajoutées au rapport (hors de ce module) |

# 2. Environnement de travail

### 2.1 Organisation des dossiers

Tout le projet est sur le disque **D:**. Le disque C: était presque plein (1,9 Go libres) et l'ancien dossier se trouvait dans OneDrive, qui aurait synchronisé des dizaines de Go de photos vers le cloud.

```
D:\skin_ai\
├── code\                      ← code source (à ouvrir dans VS Code)
│   ├── skin_analyzer\         ← module d'analyse (utilisé par le miroir)
│   │   ├── config.py          ← tous les paramètres réglables
│   │   ├── face_regions.py    ← points du visage + zones de peau
│   │   ├── detectors.py       ← détecteurs classiques (acné, rides, taches, pores)
│   │   ├── dl_models.py       ← inférence des modèles profonds (ONNX)
│   │   ├── analyzer.py        ← orchestration + scores + contrôle qualité
│   │   ├── visualize.py       ← image annotée + panneau de scores
│   │   ├── main.py            ← programme en ligne de commande (photo / webcam)
│   │   └── models\            ← modèles (points du visage, YOLO, U-Net)
│   ├── training\              ← scripts d'entraînement
│   │   ├── paths.py, download_data.py
│   │   ├── prepare_acne_data.py, train_acne.py
│   │   ├── train_wrinkles.py
│   │   ├── wsl_setup.sh, wsl_train.sh   ← installation / lancement sous Ubuntu
│   │   ├── wsl_queue.sh       ← enchaîne les rides après l'acné
│   │   └── status.py          ← avancement des entraînements
│   ├── status.bat             ← double-clic : état des entraînements
│   ├── tests\test_synthetic.py
│   └── docs\                  ← ce guide
├── datasets\                  ← bases (ACNE04, FFHQ-Wrinkle)
├── runs\                      ← résultats d'entraînement (courbes, poids, scores)
└── wsl\                       ← disque virtuel d'Ubuntu (PyTorch, copie des données)
```

## 2.2 Logiciels

| Élément | Version / choix | Justification |
|---|---|---|
| Python | 3.11 | Même version que Raspberry Pi OS (Bookworm) : le code tourne à l'identique sur le Pi |
| OpenCV (contrib) | 5.0 | Traitement d'image, détection de visage YuNet, points LBF, lecture des modèles ONNX |
| MediaPipe | 1.0 | 478 points du visage (moteur principal, sur le Pi) |
| PyTorch + CUDA 12.6 | 2.x (sous WSL2) | Entraînement sur la carte graphique NVIDIA RTX 2070 (8 Go) |
| Ubuntu 24.04 (WSL2) | Python 3.12 | Environnement Linux d'entraînement, non concerné par Smart App Control |
| Ultralytics | 8.x | Entraînement YOLO11 (détection de l'acné) |
| segmentation-models-pytorch | 0.5 | Architecture U-Net avec encodeur pré-entraîné ImageNet |
| albumentations | 2.x | Augmentation de données pour la segmentation |

## 2.3 Problèmes rencontrés et solutions

| Problème | Cause | Solution |
|---|---|---|
| MediaPipe refuse de se charger (WinError 4551) | Windows Smart App Control bloque sa DLL non signée | Second moteur de points 100 % OpenCV (YuNet + Facemark LBF, 68 points), choisi automatiquement. Sur le Raspberry Pi, MediaPipe fonctionne. |
| Disque C: plein | 1,9 Go libres ; PyTorch ≈ 5 Go, bases ≈ 20 Go | Tout déplacé vers D:\skin_ai ; cache pip purgé (1,4 Go) |
| Projet dans OneDrive | Synchronisation des Go de données vers le cloud | Données hors OneDrive (D:\skin_ai\datasets) |
| Base FFHQ complète = 65 Go | Les 70 000 photos ne sont pas toutes utiles | Téléchargement image par image des seules photos annotées (miroir Hugging Face) |
| PyTorch refuse de se charger (WinError 4551 sur c10.dll) | Même cause : Smart App Control (état « activé » dans le registre) | **Solution retenue :** entraînement sous Ubuntu 24.04 via WSL2, installé sur D:. Smart App Control reste actif sous Windows, et la RTX 2070 reste accessible depuis Linux |

## 2.4 Installation

Deux environnements Python coexistent :

- **Windows** (venv du projet) : application d'analyse, préparation des données, génération de ce guide. Pas de PyTorch nécessaire : les modèles entraînés sont lus par OpenCV.
- **Ubuntu 24.04 sous WSL2** (nommé `skinai`, stocké dans D:\skin_ai\wsl) : entraînement des modèles avec PyTorch et la carte graphique. Les données y sont copiées (/data/skin_ai/datasets) pour une lecture rapide ; les résultats sont écrits sur D: (D:\skin_ai\runs).

```
:: 1. Installer Ubuntu sur D: (une seule fois, depuis PowerShell)
wsl --install -d Ubuntu-24.04 --location D:\skin_ai\wsl --name skinai --no-launch

:: 2. Installer PyTorch (CUDA 12.6), YOLO, U-Net... dans Ubuntu
wsl -d skinai -u root -- bash /mnt/d/skin_ai/code/training/wsl_setup.sh

:: 3. Copier les données dans le disque Linux (lecture plus rapide)
wsl -d skinai -u root -- rsync -a /mnt/d/skin_ai/datasets/acne_yolo /data/skin_ai/datasets/
wsl -d skinai -u root -- rsync -a /mnt/d/skin_ai/datasets/ffhq_wrinkle /data/skin_ai/datasets/

:: 4. Vérifier la carte graphique sous Linux
wsl -d skinai -u root -- nvidia-smi
```

Le script `training/wsl_train.sh` règle les variables d'environnement (emplacement des données, des résultats et des caches) puis lance l'entraînement demandé.

# 3. Phase 1 : module d'analyse (vision classique)

Cette phase construit toute la chaîne d'analyse. Elle reste utile après la phase 2 : les modèles profonds remplacent seulement les détecteurs d'acné et de rides, et la localisation du visage, les zones, les scores et l'affichage restent les mêmes.

## 3.1 config.py : paramètres

Tous les seuils sont regroupés dans un seul fichier pour faciliter la calibration. Idée clé : **toutes les tailles sont exprimées dans une image normalisée** où la distance entre les coins externes des yeux vaut 400 px (≈ 90 mm sur un adulte, soit ≈ 4,4 px/mm). Les seuils ne dépendent donc pas de la distance entre le visage et la caméra.

| Paramètre | Valeur | Rôle |
|---|---|---|
| NORM_EYE_DIST / EYE_DIST_MM | 400 px / 90 mm | Échelle commune : conversion pixels → millimètres |
| SKIN_COLOR_TOLERANCE | 16 | Écart de couleur Lab max. avec la peau des joues (rejette cheveux, fond) |
| MIN_EYE_DIST_RAW | 140 px | En dessous : visage trop loin, avertissement |
| ACNE_RED_DELTA | 7 | Excès de rougeur a* par rapport à la peau voisine |
| WRINKLE_THRESHOLD | 0,035 | Réponse minimale du filtre de crêtes |
| SPOT_DARK_DELTA | 9 | Assombrissement L* minimal d'une tache |
| PORE_THRESHOLD | 12 | Contraste minimal d'un pore (niveaux de gris) |
| SCORE_SCALE | par critère | Échelle de conversion mesure → score 0-100 |

## 3.2 face_regions.py : points du visage et zones de peau

### Deux moteurs de points

- **MediaPipe Face Landmarker** : 478 points en 3D, très précis. Moteur principal, utilisé sur le Raspberry Pi.
- **OpenCV YuNet + Facemark LBF** : YuNet (réseau léger, 230 Ko) trouve le visage, puis LBF place 68 points (schéma iBUG : mâchoire 0-16, sourcils 17-26, nez 27-35, yeux 36-47, bouche 48-67). Moteur de secours.

La fonction `make_backend()` essaie MediaPipe et bascule automatiquement sur OpenCV en cas d'erreur. Chaque moteur décrit ses zones par des listes d'indices de points (dictionnaires `MEDIAPIPE_LAYOUT` et `LBF68_LAYOUT`) : le reste du code ne dépend pas du moteur.

### Étapes de extract()

- **1. Recadrage** autour des points, avec une marge plus grande en haut pour le front.
- **2. Mise à l'échelle** pour que la distance entre les yeux vaille 400 px.
- **3. Zones** : front, joue gauche/droite, nez, menton (polygones de points, enveloppe convexe) et pattes d'oie (rectangle orienté construit à partir du coin externe de l'œil). Avec 68 points, le front n'a pas de points : il est extrapolé au-dessus des sourcils, sur 0,4 × la distance entre les yeux.
- **4. Exclusions** : yeux, sourcils et lèvres (élargis de quelques pixels) sont retirés de toutes les zones.
- **5. Masque couleur de peau** : la couleur médiane des joues sert de référence ; tout pixel trop éloigné dans l'espace L*a*b* (cheveux, frange, fond) est retiré. La luminance L* compte pour moitié, pour tolérer les variations d'éclairage.

```
def _skin_color_mask(img, reference):
    lab = cv2.cvtColor(cv2.GaussianBlur(img, (0, 0), 2), cv2.COLOR_BGR2LAB).astype(np.float32)
    ref = np.median(lab[reference > 0], axis=0)      # couleur typique des joues
    diff = lab - ref
    diff[..., 0] *= 0.5                              # tolérance sur la luminosité
    dist = np.sqrt((diff ** 2).sum(axis=2))
    return (dist < config.SKIN_COLOR_TOLERANCE)      # True = pixel de peau
```

## 3.3 detectors.py : les quatre détecteurs classiques

> **Comparaison locale**
> Principe commun : chaque pixel est comparé à la **peau voisine**, et non à un seuil absolu. La peau voisine est estimée par un flou gaussien calculé uniquement sur les pixels de peau (fonction `masked_blur` = flou(canal × masque) / flou(masque)). La détection devient ainsi peu sensible à l'éclairage global et à la carnation.

### Acné : detect_acne()

- Canal **a*** (axe vert-rouge de L*a*b*) : un bouton est plus rouge que la peau autour.
- Δa = a*(lissé) − a*(peau voisine, σ = 6 mm) ; pixels retenus si Δa > 7.
- Ouverture morphologique (supprime le bruit), puis contours ; filtres : surface entre 1,5 et 60 mm², circularité $4\pi A/P^2 \geq 0{,}35$.
- Sortie : nombre de lésions, position, rayon, rougeur moyenne, zone.

### Rides : detect_wrinkles()

- Luminance relative = L / L(peau voisine) : vaut 1 sur une peau uniforme, moins dans un sillon.
- **Filtre de crêtes de Sato** (valeurs propres de la matrice hessienne, échelles σ = 1,5 ; 2,5 ; 3,5 px) : réagit aux structures fines, sombres et allongées.
- Une bande au bord de chaque zone est ignorée (effet de bord, cheveux).
- Composantes connexes conservées si longueur du squelette ≥ 4 mm et allongement ≥ 3.
- Mesure : mm de rides par cm² de zone (front + pattes d'oie).

### Taches : detect_spots()

- Assombrissement ΔL = L(peau voisine, σ = 8 mm) − L(lissé) > 9, **sans** excès de rouge (Δa < 6), ce qui distingue une tache pigmentaire d'un bouton.
- Surface entre 2 et 150 mm², allongement ≤ 4 (sinon c'est plutôt une ride ou une ombre).
- Mesure : pourcentage de surface de peau couverte.

### Pores / trous : detect_pores()

- **Transformée black-hat** (fermeture − image, disque de 9 px) : fait ressortir les petits points plus sombres que leur entourage.
- Points conservés : de 3 px à 1,2 mm², à peu près ronds ; zones joues et nez.
- Mesure : pores par cm². Un pore situé dans un bouton ou une tache n'est pas compté deux fois (fonction `_drop_pores_inside_lesions` de analyzer.py).

### Conversion en score

Chaque mesure brute v devient un score de sévérité de 0 à 100 par une saturation progressive : **score = 100 × (1 − $e^{-v / \text{échelle}}$)**. Niveaux : faible (< 25), modéré (< 55), élevé. Le score global de peau vaut 100 − (0,35·acné + 0,25·rides + 0,20·taches + 0,20·pores).

## 3.4 analyzer.py, visualize.py, main.py

- **analyzer.py** : classe `SkinAnalyzer` ; méthode `analyze(image)` qui renvoie le rapport et les zones. Contrôle qualité : distance entre les yeux, luminosité moyenne de la peau, netteté (variance du Laplacien). `report_to_json()` retire les données lourdes pour l'envoi (MQTT, serveur web).
- **visualize.py** : dessine les zones, les boutons (cercles rouges), les taches (rectangles orange), les rides (surlignées en bleu) et les pores (points jaunes), plus un panneau latéral de scores.
- **main.py** : programme en ligne de commande, sur une photo ou en direct avec la webcam (ESPACE = analyser, Z = zones, Q = quitter). Les résultats sont enregistrés dans results/.

```
python -m skin_analyzer.main --image photo.jpg
python -m skin_analyzer.main --camera 0
python -m skin_analyzer.main --camera 0 --backend opencv
```

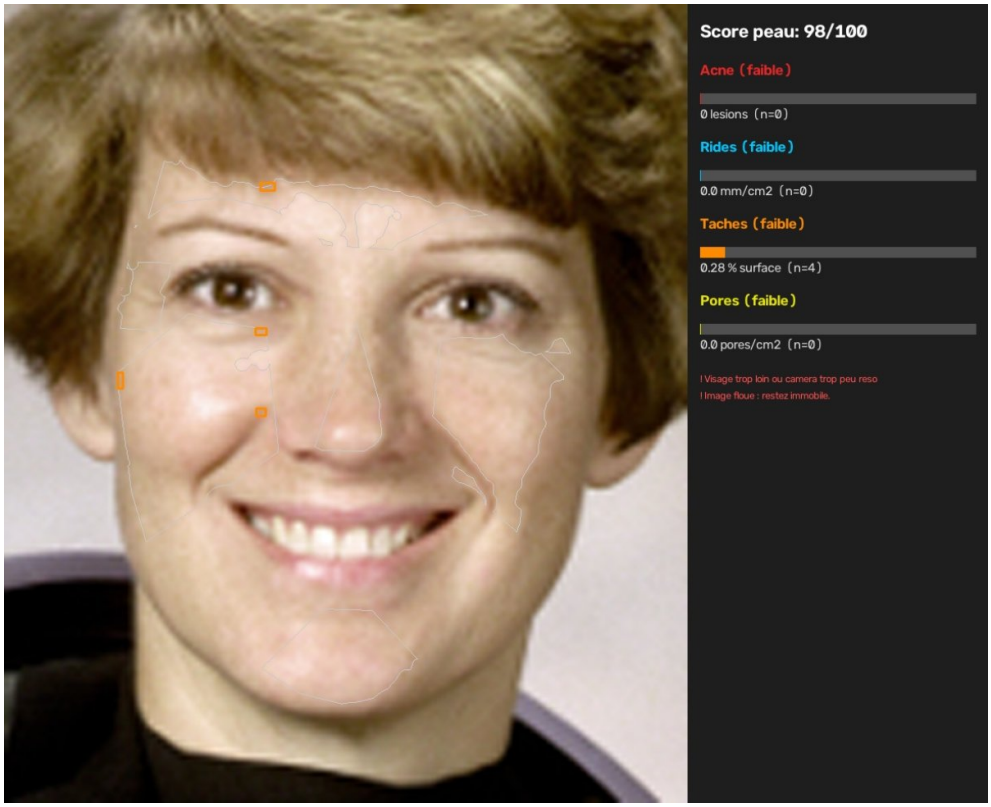



*Figure 2 : analyse classique d'une photo libre de droits (zones en gris, taches en orange, panneau de scores à droite).*

## 3.5 tests/test_synthetic.py : test par injection

Faute de photos annotées pour la vision classique, on **dessine de faux défauts** de taille réaliste sur un vrai visage, puis on vérifie que chaque détecteur les retrouve à leur position :

- 6 disques rouges de ≈ 3 mm sur les joues (acné) ;
- 3 disques brun foncé de ≈ 5 mm (taches), espacés des boutons ;
- 3 lignes sombres sur le front et les pattes d'oie (rides) ;
- 25 petits points sombres sur le nez (pores).

| Critère | Défauts retrouvés | Résultat |
|---|---|---|
| Acné | 100 % (6/6) | OK |
| Taches | 100 % (3/3) | OK |
| Pores | 100 % (25/25) | OK |
| Rides | 64 % des pixels de rides dessinés | OK (seuil 60 %) |

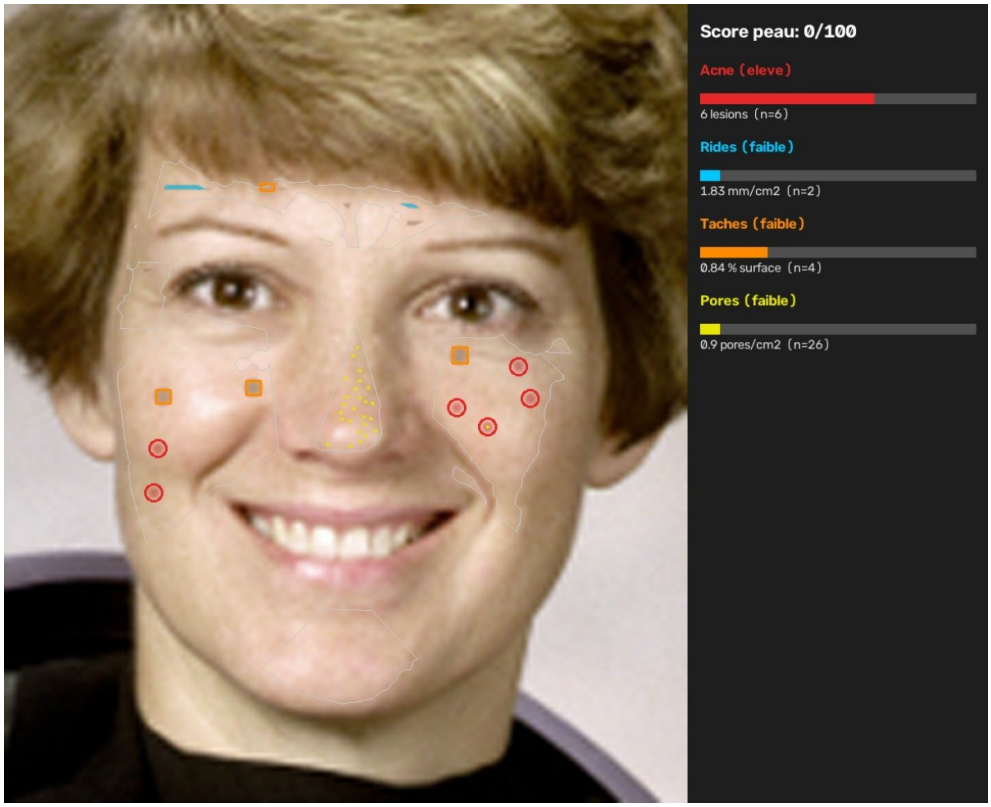



*Figure 3 : test par injection ; les défauts dessinés sont tous retrouvés.*

> **Limite**
> Ce test prouve que les détecteurs fonctionnent comme prévu, mais **pas** qu'ils sont précis sur de vraies lésions. C'est la raison de la phase 2 : de vrais modèles évalués sur de vraies annotations humaines.

# 4. Phase 2 : bases de données d'apprentissage

## 4.1 Bases étudiées et choix

| Base | Contenu | Décision |
|---|---|---|
| **ACNE04**<br>Wu et al., ICCV 2019 | 1 457 photos de visages, **18 983 lésions** d'acné annotées en boîtes par des dermatologues ; 4 niveaux de sévérité | **Retenue pour l'acné.** Base de référence de la littérature ; usage académique libre |
| **FFHQ-Wrinkle**<br>Moon et al., ICPR 2024 | 1 000 masques de rides tracés à la main + 50 000 masques « faibles » automatiques, sur les photos FFHQ 1024×1024 | **Retenue pour les rides.** Première base publique de segmentation des rides du visage entier ; CC BY-NC-SA 4.0 |
| Skin-Problem-Detection<br>(Roboflow) | 2 660 images, 10 classes (acné, taches, pores, rides...) | Écartée : images de 255×255 px (pores invisibles), annotations incohérentes (« Eyebags » encadrant tout le visage, rides en grosses boîtes) |



*Figure 4 : exemples FFHQ-Wrinkle ; à droite, les rides annotées à la main (en rouge). Elles couvrent en moyenne 0,7 % des pixels.*

## 4.2 training/download_data.py

- **ACNE04** : téléchargement du dossier Google Drive officiel avec `gdown`, puis extraction des archives Classification.tar (photos) et Detection.tar (annotations VOC XML).
- **FFHQ-Wrinkle** : masques manuels et masques faibles depuis Google Drive ; les photos FFHQ correspondantes sont prises une par une sur le miroir Hugging Face `marcosv/ffhq-dataset` (dossier Part{numéro // 10000 + 1}), avec 8 téléchargements en parallèle et reprise automatique.
- Variables : `SKIN_AI_DIR` (défaut D:/skin_ai) et `N_WEAK` (nombre d'images pour le pré-entraînement, défaut 10 000, soit ≈ 13 Go).

```
cd /d D:\skin_ai\code
..\venv\Scripts\python -m training.download_data acne
..\venv\Scripts\python -m training.download_data wrinkles
```

# 5. Modèle d'acné : YOLO11 sur ACNE04

## 5.1 prepare_acne_data.py : préparation en tuiles

Dans ACNE04, un bouton ne mesure que 0,5 à 2 % de la largeur de la photo. Si l'on réduit toute la photo à 1024 px, les plus petits ne font plus que quelques pixels et deviennent indétectables. La solution est le **découpage en tuiles** :

- tuiles de 1024×1024 px à la **résolution d'origine**, chevauchement de 25 % ;
- une boîte est gardée dans une tuile si au moins 50 % de sa surface y est visible ;
- 30 % des tuiles sans bouton sont conservées comme **exemples négatifs** (limite les fausses détections) ;
- la photo entière réduite à 1024 px est ajoutée (contexte global du visage).

Résultat obtenu (photos de 3112×3456 à 5184×3456 px) :

| Partie | Photos | Lésions annotées | Tuiles générées | Boîtes dans les tuiles |
|---|---|---|---|---|
| Entraînement | 1 166 | 14 813 | 12 369 | 42 042 |
| Validation | 146 | 2 007 | 1 569 | 5 723 |
| Test | 145 | 2 163 | 1 508 | 5 883 |

Il y a plus de boîtes que de lésions parce qu'une même lésion apparaît dans plusieurs tuiles qui se chevauchent, et dans la photo entière réduite.

> **Point méthodologique important**
> Le découpage train / validation / test (80 / 10 / 10 %) se fait **par photo, avant le découpage en tuiles**. Deux tuiles d'une même photo ne peuvent donc jamais se retrouver l'une en entraînement et l'autre en test, ce qui gonflerait artificiellement les scores.

## 5.2 train_acne.py : entraînement

| Hyperparamètre | Valeur | Justification |
|---|---|---|
| Modèle | YOLO11s (9,4 M param.) | Seule taille qui tient en 1024 px avec un lot raisonnable sur 8 Go (voir 5.3) |
| Poids initiaux | pré-entraînés COCO | Apprentissage par transfert : convergence plus rapide, meilleure précision |
| Taille d'entrée | 1024 px | Préserve les petites lésions |
| Epochs / patience | 150 / 40 | Entraînement long, arrêt automatique sans progrès pendant 40 epochs |
| Optimiseur | AdamW, lr 0,001, cosinus | Stable ; décroissance douce du taux d'apprentissage |
| Batch | 8 images (≈ 5 Go) | Mesuré sur la RTX 2070 ; précision mixte (AMP) ; gradients accumulés jusqu'à 64 |
| Augmentation couleur | HSV h 0,015 · s 0,5 · v 0,4 | Robustesse à l'éclairage et à la caméra du miroir |
| Augmentation géométrique | rotation ±10°, échelle ±50 %, miroir | Pose et distance variables |
| Mosaic / mixup | 1,0 / 0,1 ; arrêt mosaic 15 dernières epochs | Plus d'exemples variés, fin d'entraînement sur images réalistes |

En fin d'entraînement, le script évalue automatiquement le meilleur modèle sur la **partie test** (photos jamais vues), enregistre les scores dans `test_metrics.json`, puis exporte le modèle en ONNX vers `skin_analyzer/models/acne_yolo.onnx`.

```
python -m training.prepare_acne_data                    :: sous Windows
set T=wsl -d skinai -u root -- bash /mnt/d/skin_ai/code/training/wsl_train.sh
%T% acne --epochs 3 --name essai                        :: essai rapide
%T% acne                                                :: entraînement complet
%T% acne --resume                                       :: reprise après coupure
```

## 5.3 Choix de la taille du modèle : mesures réelles

Le premier choix était YOLO11m, plus gros et en principe plus précis. L'essai a montré qu'il ne tenait pas dans les 8 Go de la RTX 2070 en 1024 px. On a alors privilégié la résolution, indispensable pour des boutons de quelques pixels, plutôt que la taille du réseau :

| Configuration (1024 px) | Mémoire GPU | Vitesse | Durée estimée (150 epochs) | Décision |
|---|---|---|---|---|
| YOLO11m, lot de 2 | 6,8 Go | ≈ 26 min / epoch | ≈ 65 h | Écartée (trop long) |
| YOLO11m, lot de 4+ | > 8 Go | - | - | Impossible (mémoire saturée) |
| **YOLO11s, lot de 8** | **≈ 5 Go** | **≈ 6 min / epoch** | **≈ 15 h max** | **Retenue** |

Réduire l'image à 640 px pour garder YOLO11m aurait fait perdre les plus petites lésions : à résolution égale, un modèle plus petit détecte mieux ce qu'il voit qu'un modèle plus gros qui ne voit plus les détails.

## 5.4 Métriques d'évaluation

| Métrique | Définition | Lecture |
|---|---|---|
| Précision | VP / (VP + FP) | Parmi les boutons détectés, part de vrais boutons |
| Rappel | VP / (VP + FN) | Parmi les vrais boutons, part de boutons trouvés |
| mAP@50 | Aire sous la courbe précision-rappel, IoU ≥ 0,5 | Métrique principale en détection |
| mAP@50-95 | Moyenne des mAP pour IoU de 0,5 à 0,95 | Exige des boîtes très bien placées |

| Résultat sur le test ACNE04 (145 photos jamais vues) | Valeur |
|---|---|
| precision | 0.439 |
| rappel | 0.400 |
| mAP50 | 0.337 |
| mAP50-95 | 0.089 |

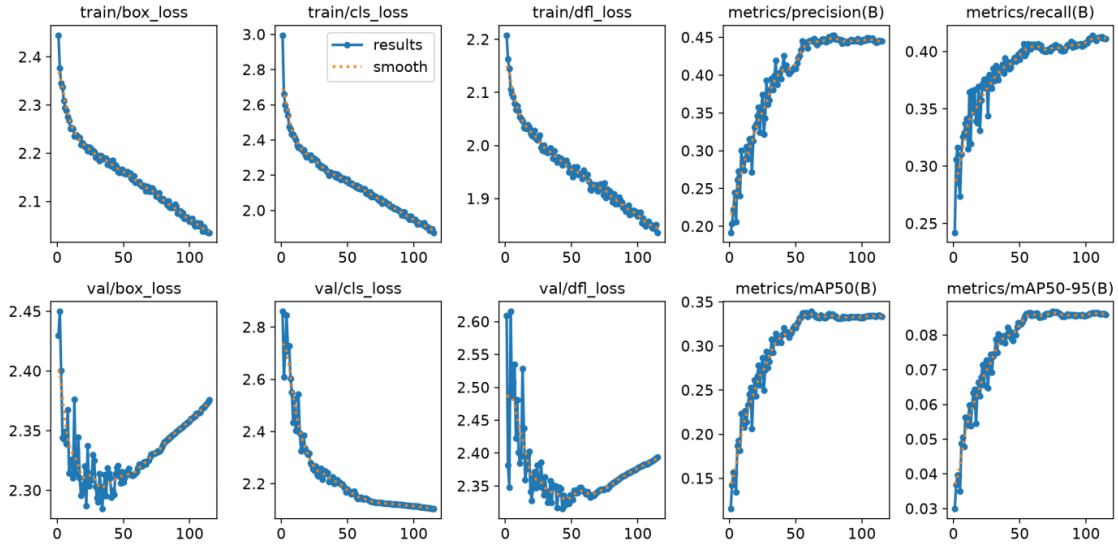



*Figure 5 : courbes d'entraînement YOLO11s. Les métriques de validation plafonnent vers l'epoch 60 ; l'arrêt automatique intervient à l'epoch 115.*




*Figure 6 : prédictions sur le test (confiance affichée ≥ 0,25). Le modèle trouve les lésions les plus nettes mais en manque beaucoup sur les acnés sévères.*

**Analyse du résultat**
Le score est en dessous de ce qui était espéré (mAP@50 ≈ 0,4-0,6). Les confiances ne dépassent jamais 0,5 : le modèle hésite, signe d'annotations difficiles (lésions minuscules et très denses). Le seuil de confiance de l'application est fixé à **0,16**, valeur qui maximise le F1 sur la **validation** (0,41). Le test mélange tuiles et photos entières réduites, alors que l'application n'utilise que des tuiles : le score réel en utilisation est probablement un peu meilleur. Pistes d'amélioration : retirer les photos réduites de l'entraînement, tuiles plus petites agrandies.

**Évaluation par photo, dans l'application** (analyse complète par tuiles, comme sur le miroir) : sur les 125 photos de test où un visage est détecté (21 profils trop marqués sont rejetés), le nombre de

boutons détectés suit le nombre annoté avec une **corrélation de 0,75**, mais le modèle en trouve environ la moitié (6,1 en moyenne contre 11,5 annotés ; erreur moyenne de 6 boutons). Le classement des niveaux de sévérité est donc fiable, mais pas le compte exact : l'échelle du score d'acné en tient compte (3,5 boutons détectés au lieu de 6).

# 6. Modèle de rides : U-Net sur FFHQ-Wrinkle

## 6.1 Pourquoi la segmentation ?

Une ride est une ligne fine et souvent courbe : une boîte englobante la décrit très mal. La **segmentation sémantique** prédit, pour chaque pixel, la probabilité qu'il appartienne à une ride. On obtient la forme exacte, d'où l'on déduit la longueur et la densité par zone.

## 6.2 train_wrinkles.py : méthode en deux étapes

La méthode reprend celle de l'article FFHQ-Wrinkle :

- **Étape 1, pré-entraînement** sur les masques « faibles » (≈ 10 000 images, 8 epochs, lr $3 \cdot 10^{-4}$). Ces masques, générés automatiquement à partir de cartes de texture, sont imparfaits, mais leur nombre apprend au réseau ce qu'est une texture de ride.
- **Étape 2, ajustement fin** sur les 800 masques manuels d'entraînement (120 epochs, lr $10^{-4}$), avec évaluation sur la validation toutes les 2 epochs et sauvegarde du meilleur modèle (IoU maximale).

| Choix | Valeur | Justification |
|---|---|---|
| Architecture | U-Net, encodeur ResNet-50 (ImageNet) | Connexions de saut : conserve les détails fins ; encodeur pré-entraîné |
| Résolution | crops de 512 px pris dans des images de 1024 px | Pas de réduction : une ride fait 3 à 6 px de large |
| Échantillonnage | 1 recadrage sur 2 centré sur une zone contenant des rides | Les rides étant rares (0,7 % des pixels), le réseau en voit assez |
| Perte | Dice + 2 × Focal (α 0,75 ; γ 2) | Adaptée aux classes très déséquilibrées (0,7 % de pixels) |
| Augmentation | miroir, rotation ±12°, échelle ±25 %, luminosité, teinte, bruit, JPEG, flou | Robustesse à la caméra et à l'éclairage du miroir |
| Optimisation | AdamW + OneCycle, précision mixte | Convergence rapide et stable sur 8 Go |
| Test-time augmentation | moyenne image + image miroir | Prédiction plus stable |

> **Protocole d'évaluation**
> Les 1 000 masques manuels sont répartis une fois pour toutes en 800 / 100 / 100 (fichier `split.json`, graine fixe). Le **seuil de décision** est choisi sur la validation, puis appliqué tel quel au test. Le test n'influence donc aucun choix et les scores obtenus sont honnêtes.

La perte combine deux termes. **Dice** = $1 - 2|P \cap G| / (|P| + |G|)$ mesure le chevauchement global entre prédiction P et vérité G, sans être dominée par le fond. **Focal** = $-\alpha(1-p)^{\gamma}\log(p)$ donne plus de poids aux pixels difficiles (bords de rides, ridules).

```
%T% wrinkles                        :: étapes 1 + 2 + test
%T% wrinkles --stage finetune       :: étape 2 seule
%T% wrinkles --stage test           :: évaluation seule
```

## 6.3 Métriques

| Métrique | Définition |
|---|---|
| IoU (indice de Jaccard, JSI) | VP / (VP + FP + FN), calculé sur tous les pixels du test |
| F1 (Dice) | 2·VP / (2·VP + FP + FN) |
| Précision / rappel | au niveau du pixel |

| Résultat sur le test FFHQ-Wrinkle (100 visages jamais vus) | Valeur |
|---|---|
| IoU(JSI) | 0.4746 |
| F1(Dice) | 0.6437 |
| precision | 0.6277 |
| rappel | 0.6606 |
| seuil | 0.55 |



*Figure 7 : à gauche les rides annotées à la main (rouge), à droite les rides prédites par le modèle exporté, exécuté dans l'application (OpenCV, alignement FFHQ).*

**Analyse du résultat**
L'étape 1 (masques faibles) donne une IoU quasi nulle sur les masques manuels : les deux types d'annotations sont très différents. L'ajustement fin fait ensuite monter l'IoU de validation de 0,29 (epoch 4) à 0,49 (epoch 112). Dans l'application, l'IoU par image est un peu plus basse (≈ 0,37) car le recadrage refait à partir des points du visage décale les traits d'un ou deux pixels, ce qui pèse lourd pour des lignes aussi fines. Temps de calcul : ≈ 3,3 s par visage sur le processeur du PC (deux passages, image + miroir).

**Étalonnage du score :** sur les 100 visages de test, l'application mesure en médiane 0,9 mm de rides par cm² de visage (25 % : 0,58 ; 75 % : 1,31 ; 90 % : 2,32). L'échelle du score est fixée à 2,1 : un visage médian obtient 35 (modéré), les 10 % les plus ridés dépassent 65 (élevé).

# 7. Intégration dans le miroir : dl_models.py

Les modèles entraînés sont exportés au format **ONNX** et lus par **OpenCV DNN**. Il n'y a aucune dépendance supplémentaire, que ce soit sur le PC Windows ou sur le Raspberry Pi.

## 7.1 Rides : alignement FFHQ

Le modèle de rides a été entraîné sur des visages cadrés d'une manière bien précise (alignement FFHQ). À l'utilisation, on reproduit **exactement le même cadrage** : un quadrilatère est calculé à partir du centre des yeux et des coins de la bouche, puis le visage est redressé en 1024×1024 par une transformation affine. Le masque prédit est ensuite ramené dans l'image d'origine par la transformation inverse.

```
eye_avg = (eye_l + eye_r) / 2 ;  eye_to_eye = eye_r - eye_l
eye_to_mouth = (mouth_l + mouth_r) / 2 - eye_avg
x = eye_to_eye - flipud(eye_to_mouth) * [-1, 1]
x = x / |x| * max(2.0 * |eye_to_eye|, 1.8 * |eye_to_mouth|)
y = flipud(x) * [-1, 1] ;  c = eye_avg + 0.1 * eye_to_mouth
quad = [c - x - y, c - x + y, c + x + y, c + x - y]      # même formule que NVlabs
```

## 7.2 Acné : inférence par tuiles

- Comme à l'entraînement, l'image est parcourue par tuiles de 1024 px qui se chevauchent (ajout de bordure grise pour les tuiles incomplètes).
- Sortie YOLO décodée : (cx, cy, w, h, score) pour chaque candidat.
- Les boîtes de toutes les tuiles sont fusionnées par **suppression des non-maxima** (NMS, IoU 0,45), ce qui retire les doublons dans les zones de chevauchement.
- Seules les détections dont le centre tombe sur la peau (zones du visage) sont gardées.

# 8. Procédure complète pas à pas

| # | Étape | Commande (depuis D:\skin_ai\code) | Durée indicative |
|---|---|---|---|
| 1 | Installer Ubuntu (WSL2) + PyTorch | voir section 2.4 | 20-30 min |
| 2 | Télécharger ACNE04 (Windows) | python -m training.download_data acne | 10-20 min |
| 3 | Télécharger FFHQ-Wrinkle (Windows) | python -m training.download_data wrinkles | 1-4 h (≈ 22 Go) |
| 4 | Préparer les tuiles d'acné (Windows) | python -m training.prepare_acne_data | ≈ 5 min |
| 5 | Copier les données dans Ubuntu | voir section 2.4, étape 3 | ≈ 15 min |
| 6 | Essai rapide YOLO (Ubuntu) | %T% acne --epochs 3 --name essai | ≈ 20 min |
| 7 | Entraîner l'acné (Ubuntu) | %T% acne | ≈ 6-12 h (RTX 2070) |
| 8 | Entraîner les rides (Ubuntu) | %T% wrinkles | ≈ 8-14 h (RTX 2070) |
| 9 | Tester sur une photo (Windows) | python -m skin_analyzer.main --image photo.jpg | secondes |

| # | Étape | Commande (depuis D:\skin_ai\code) | Durée indicative |
|---|---|---|---|
| 10 | Mettre ce guide à jour (Windows) | python docs\build_guide.py | secondes |

`%T%` désigne `wsl -d skinai -u root -- bash /mnt/d/skin_ai/code/training/wsl_train.sh`. Les commandes Windows se lancent depuis D:\skin_ai\code avec l'environnement Python du projet.

### 8.1 Lancer et suivre un entraînement long

- Chaque entraînement est lancé dans **sa propre fenêtre** (ne pas la fermer). Le script `wsl_queue.sh` attend la fin de l'acné puis démarre les rides tout seul, car la carte graphique ne peut pas faire les deux à la fois.
- **status.bat** (double-clic) affiche pour chaque modèle : état (pas démarré / EN COURS / TERMINÉ / ARRÊTÉ ?), epoch en cours, temps restant estimé, meilleur score de validation, puis les scores sur le test une fois terminé.
- Journaux complets : `D:\skin_ai\runs\acne_train.log` et `wrinkles_train.log` ; la dernière ligne `=== Fin ... code=0` signifie succès.
- Le fichier `test_metrics.json` n'apparaît qu'à la toute fin : sa présence indique que l'entraînement est terminé.
- Le PC ne doit pas se mettre en veille pendant l'entraînement.

> **Pour le rapport**
> Les courbes d'apprentissage (perte, précision, rappel, mAP), la matrice de confusion et des exemples de prédictions sont générés automatiquement dans `D:\skin_ai\runs\`. Ce sont de bonnes figures pour le rapport.

## 9. Partie matérielle : miroir, ESP32 et Raspberry Pi

Le miroir réunit deux cartes aux rôles bien séparés : l'**ESP32** s'occupe des capteurs et de la LED (temps réel, électronique), le **Raspberry Pi 4** de la caméra, de l'analyse IA, de l'affichage et de l'historique. Ils communiquent par le câble USB-C, qui alimente aussi l'ESP32.

### 9.1 Rôle de chaque composant

| Composant | Rôle | Connexion |
|---|---|---|
| Raspberry Pi 4 | Caméra, analyse (modèles ONNX), interface, historique | Alimentation 5 V / 3 A |
| Pi Camera Module 3 | Photo du visage 12 MP, mise au point automatique | Nappe CSI du Pi |
| Écran HDMI 5" (800×480) | Interface sur fond noir, derrière le miroir | HDMI du Pi |
| ESP32 | Lit les capteurs, pilote la LED, envoie du JSON 5 fois/s | USB-C → USB du Pi |
| VL53L0X / VL53L1X | Distance du visage : déclenche et guide l'analyse | I2C (GPIO21/22) |
| MLX90614 | Température de la peau sans contact | I2C (GPIO21/22) |
| DHT22 | Température et humidité de l'air (conseils d'hydratation) | GPIO4 |
| LED RGB | État du miroir : attente, détection, analyse, OK, erreur | GPIO25/26/27 + 220 Ω |

## 9.2 Carte retenue : Arduino Uno + capteur Sharp IR

La partie capteurs a été réalisée par J. Salek (dépôt `jihanesalek263-png/SmartSkinAnalyzer`) sur un **Arduino Uno** avec un capteur de distance **Sharp GP2Y0A21** (analogique, 10-80 cm). C'est la carte retenue pour le miroir. Le firmware `firmware/uno_capteurs/uno_capteurs.ino` reprend le sien avec le **même format JSON**, en ajoutant un envoi 4 fois par seconde (au lieu de toutes les 2 s, pour guider la personne en direct) et le pilotage de la LED RGB. Le miroir reconnaît automatiquement l'Uno ou l'ESP32.

| Module | Broche module | Broche Arduino Uno | Remarque |
|---|---|---|---|
| MLX90614 | VIN / GND | 3.3V / GND | jamais 5 V sur un capteur nu |
| MLX90614 | SDA / SCL | A4 / A5 | bus I2C de l'Uno |
| DHT22 | patte 1 / 2 / 4 | 5V / D2 / GND | résistance 10 kΩ entre les pattes 1 et 2 |
| Sharp GP2Y0A21 | rouge / noir / jaune | 5V / GND / A0 | distance par la tension analogique |
| LED RGB | R / G / B | D9 / D10 / D11 | 220 Ω sur chaque couleur ; patte commune au GND |

Le firmware compile pour l'Uno (40 % de la mémoire programme, 34 % de la RAM). Message envoyé : `{"distance_cm":18.3,"temp_peau":33.4,"temp_air":24.1,"humidite":46.2}`.

## 9.2 bis Variante : câblage de l'ESP32

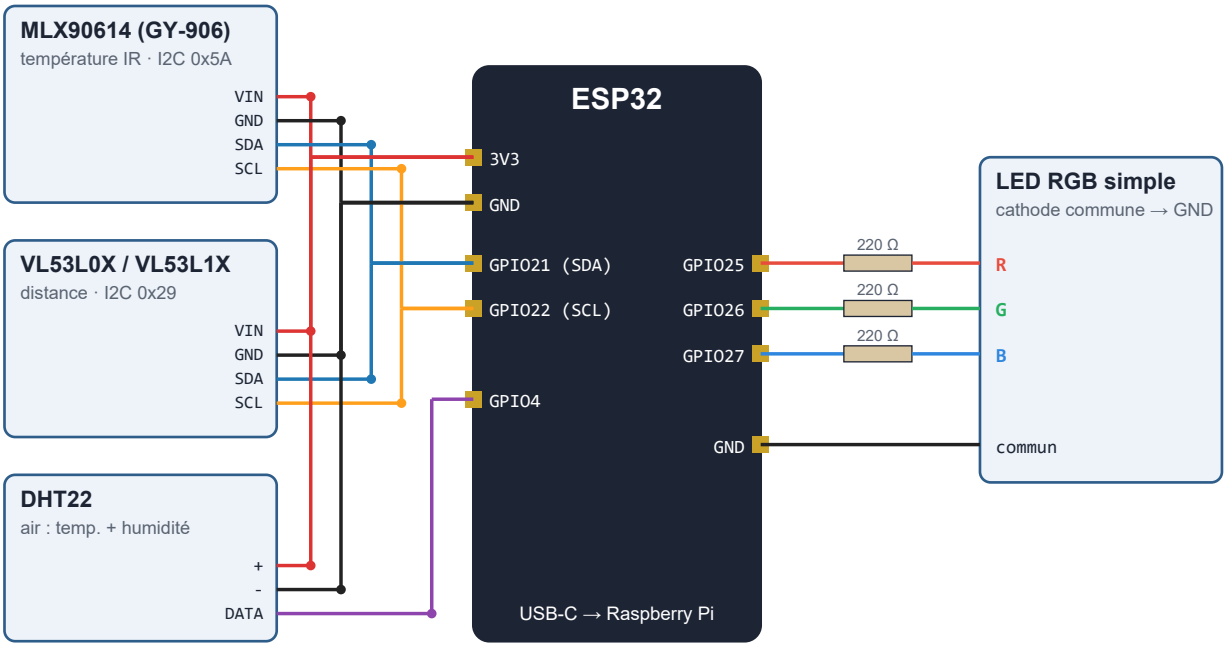



*Figure 8 : câblage. Les deux capteurs I2C partagent les mêmes fils SDA/SCL (adresses différentes : 0x5A et 0x29).*

| Module | Broche module | Broche ESP32 | Remarque |
|---|---|---|---|
| MLX90614, VL53L0X | VIN / GND | 3V3 / GND | les modules GY acceptent 3,3 V |
| MLX90614, VL53L0X | SDA / SCL | GPIO21 / GPIO22 | bus I2C partagé, 100 kHz |
| DHT22 | + / - / DATA | 3V3 / GND / GPIO4 | résistance 10 kΩ DATA→3V3 si capteur nu (pas sur module) |

| Module | Broche module | Broche ESP32 | Remarque |
|---|---|---|---|
| LED RGB | R / G / B | GPIO25 / 26 / 27 | une résistance de 220 Ω sur chaque couleur |
| LED RGB | patte commune (la plus longue) | GND (ou 3V3) | si 3V3 : mettre LED_ANODE_COMMUNE à 1 dans le firmware |

## 9.3 Montage du miroir

- **Écran derrière le miroir sans tain** (ou vitre + film miroir). L'interface utilise un fond **noir** : le noir n'émet pas de lumière, cette zone reste un miroir, et seuls les textes clairs apparaissent à travers.
- **Caméra** en haut, au centre, à hauteur des yeux. Un miroir sans tain laisse passer ≈ 30 % de la lumière : découper un petit trou dans le film en face de l'objectif donne des photos bien plus nettes.
- **Capteurs VL53L0X et MLX90614 jamais derrière le verre** : le verre est opaque à l'infrarouge thermique (MLX90614) et le capteur de distance mesurerait le verre. Les placer dans le cadre, face à l'utilisateur, à côté de la caméra.
- **MLX90614 :** le modèle courant (GY-906 « BAA ») voit sous un angle de 90°. À 40 cm il mesure la moyenne d'une large zone (visage, cheveux, fond) : la valeur est plus basse que la vraie température de la peau. La version « DCI » (angle de 5°) est plus juste à distance.
- **Éclairage :** la LED indique l'état. Pour la qualité des photos, un éclairage frontal diffus (bande LED blanche autour du miroir) réduit les ombres prises pour des taches.
- **DHT22** à l'air libre, loin de l'écran et du Pi qui chauffent.

## 9.4 Firmware de l'ESP32 (firmware/esp32_capteurs.ino)

- IDE Arduino → *Préférences* → URL de cartes : `https://espressif.github.io/arduino-esp32/package_esp32_index.json`, puis *Gestionnaire de cartes* → installer **esp32 by Espressif** (version 3.x).
- *Gestionnaire de bibliothèques* : **Adafruit MLX90614 Library**, **DHT sensor library** (+ Adafruit Unified Sensor), **VL53L0X** et **VL53L1X** (Pololu).
- Carte : **ESP32 Dev Module** ; choisir le port COM ; Téléverser. Au démarrage, la LED affiche rouge, vert puis bleu : c'est le test de câblage.
- Le firmware détecte seul le VL53L0X ou le VL53L1X et continue de fonctionner si un capteur manque (valeur `null`).

| Sens | Message | Exemple |
|---|---|---|
| ESP32 → Pi | mesure (5 fois/s) | {"type":"mesure","dist_mm":412,"t_visage":34.8,"t_air":23.6,"humidite":41.2} |
| ESP32 → Pi | présentation | {"type":"hello","capteurs":{"mlx":true,"tof":"VL53L0X","dht":true}} |
| Pi → ESP32 | état de la LED | ETAT ANALYSE (attente, detecte, analyse, ok, erreur, eteint) |
| Pi → ESP32 | couleur libre | LED 255 0 0 |

## 9.5 Installation du Raspberry Pi

- **Raspberry Pi Imager** (sur le PC) : appareil *Raspberry Pi 4*, système *Raspberry Pi OS (64-bit)* avec bureau, carte microSD. Dans les réglages : nom d'hôte `skinmirror`, utilisateur et mot de passe, WiFi, et **activer SSH**. Écrire la carte.
- Brancher caméra (nappe CSI, Pi éteint), écran HDMI, ESP32 (USB), puis l'alimentation.

- Copier le projet depuis le PC : `scp -r D:\skin_ai\code skinmirror.local:~/skin_ai` (ou une clé USB). Les modèles sont dans `skin_analyzer/models` (≈ 230 Mo).
- Sur le Pi : `cd ~/skin_ai && bash deploy/setup_pi.sh` puis `sudo reboot`. Le miroir se lance ensuite tout seul à chaque démarrage.

## 9.6 Programme du miroir (mirror/)

| Fichier | Rôle |
|---|---|
| app.py | Machine à états : attente → guidage → compte à rebours → photo → température de près → analyse → résultats |
| skin_type.py | Type de peau (sèche, normale, mixte, grasse) : règles de J. Salek, affiché « indicatif » |
| esp32.py | Lecture des mesures dans un thread, reconnexion automatique, commandes de LED |
| camera.py | Pi Camera Module 3 (picamera2, mise au point avant la photo), webcam ou photo de test |
| ui.py | Écrans sur fond noir (horloge, guidage par la distance, 3-2-1, analyse, résultats) |
| routine.py | Conseils simples selon les scores, l'humidité de l'air et la température de la peau |
| history.py | Historique (historique.jsonl) et évolution par rapport à l'analyse précédente |
| config.py | Plage de distance (30-48 cm), durées, résolution, rotation de la caméra |




*Figure 9 : écrans du miroir, capturés en mode test (personne simulée, analyse d'une photo ACNE04 par les deux modèles profonds).*

## 9.6 bis Température de la peau : mesure en 2 temps

Le MLX90614 n'est fiable qu'entre 10 et 30 cm : plus loin, il voit aussi le décor et sous-estime la température. La photo, elle, se prend à 30-48 cm pour cadrer tout le visage. Le miroir procède donc en deux temps : après la photo, il affiche « Approchez-vous à 20 cm » avec une jauge de distance, fait 4 lectures de température dans la zone fiable et garde la **médiane**, pendant que l'analyse IA de la photo continue en arrière-plan (aucun temps perdu). Sans lecture fiable au bout de 12 s, la dernière valeur est gardée et le type de peau est marqué « peu fiable ».

> **Limite du type de peau**
> Le type de peau repose sur des **règles fixées à la main** (et non sur un modèle entraîné : le jeu de données d'origine est généré par ces mêmes règles). De plus, le DHT22 mesure l'humidité de l'**air**, pas l'hydratation de la peau. Le résultat est donc affiché comme une estimation indicative ; une vraie mesure d'hydratation demanderait un capteur capacitif au contact.

### 9.7 Mise en service, étape par étape

| # | Test | Commande | Résultat attendu |
|---|---|---|---|
| 1 | ESP32 seul (sur le PC ou le Pi) | python tools/test_esp32.py | distance qui varie quand on bouge la main ; températures plausibles ; LED qui change de couleur |
| 2 | Caméra du Pi | rpicam-still -o test.jpg | photo nette du visage |
| 3 | Miroir sans capteurs | python -m mirror.app --sim --windowed | P = personne, ESPACE = analyse |
| 4 | Miroir complet | bash deploy/start_mirror.sh | s'approcher : guidage, 3-2-1, analyse, résultats |

**Ce qui reste à valider sur le matériel**
Le firmware a été compilé et le programme du miroir testé de bout en bout en simulation (sans matériel). Restent à vérifier sur le vrai matériel : les valeurs des capteurs, la netteté de la caméra derrière le miroir, et le **temps d'analyse sur le Raspberry Pi 4** (≈ 5 s sur le PC ; nettement plus sur le Pi : si c'est trop lent, passer WRINKLE_TTA à False dans skin_analyzer/config.py divise par deux le temps du modèle de rides).

## 10. Limites, éthique et suite du projet

### 10.1 Limites

- **Écart de domaine.** Les photos ACNE04 et FFHQ ne sont pas prises par la caméra du miroir. La précision réelle doit être vérifiée sur quelques photos du miroir, avec le même éclairage.
- **Taches et pores** restent en vision classique, faute de base publique fiable : leurs seuils doivent être calibrés sur des photos réelles du miroir.
- **Les pores** ne sont visibles qu'avec une bonne résolution (≥ 1080p, à 30-40 cm, lumière diffuse).
- **Biais.** La répartition des carnations, des âges et des types de peau dans les bases influence les performances ; à vérifier sur des volontaires variés.

### 10.2 Éthique et licences

- Ce système donne une **indication cosmétique**, pas un diagnostic médical.
- Photos de volontaires : consentement écrit, données anonymisées, stockage local.
- ACNE04 : usage académique uniquement (citer Wu et al., ICCV 2019).
- FFHQ-Wrinkle : CC BY-NC-SA 4.0, usage non commercial (citer Moon et al., ICPR 2024).

### 10.3 Prochaines étapes

- Mesurer le temps d'analyse sur le Raspberry Pi 4 et l'optimiser si besoin (sans double passage, export NCNN de YOLO, modèle de rides plus léger).
- Calibrer les seuils des taches et des pores sur des photos prises par la caméra du miroir.
- Améliorer le modèle d'acné (version 2 : sans photos réduites, tuiles agrandies).
- Option IoT : publier le rapport JSON en MQTT pour un tableau de bord sur téléphone.

## 10.4 Références


- X. Wu et al., « Joint Acne Image Grading and Counting via Label Distribution Learning », ICCV 2019. Base ACNE04 : github.com/xpwu95/LDL
- J. Moon et al., « Facial Wrinkle Segmentation for Cosmetic Dermatology: Pretraining with Texture Map-Based Weak Supervision », ICPR 2024. github.com/labhai/ffhq-wrinkle-dataset
- T. Karras et al., « A Style-Based Generator Architecture for GANs », CVPR 2019 (base FFHQ).
- Ultralytics YOLO11 : docs.ultralytics.com
- O. Ronneberger et al., « U-Net: Convolutional Networks for Biomedical Image Segmentation », MICCAI 2015.
- Y. Sato et al., « Three-dimensional multi-scale line filter for segmentation and visualization of curvilinear structures in medical images », Medical Image Analysis, 1998.
- Google MediaPipe Face Landmarker ; OpenCV YuNet et Facemark LBF.